Particle Swarm Optimization: How Individual Memory and Social Information Balance Exploration and Exploitation

CS420 Introduction to Artificial Intelligence · Seminar S01 · November 2026

[Member 1: full name, student ID] · [Member 2: full name, student ID] · [Member 3: full name, student ID]

Abstract

Particle Swarm Optimization (PSO) searches a continuous space with a population of particles that are pulled towards their own best positions and towards the best position found by the swarm. This report reviews the original algorithm of Kennedy and Eberhart (1995) and the later changes surveyed by Poli et al. (2007), then studies how PSO balances exploration and exploitation on two 2-D benchmark functions, the unimodal Sphere and the multimodal Rastrigin. We give a step-by-step worked example, simulate five particles over five iterations, compare three inertia weights ($\omega = 0.4, 0.7, 0.9$) over five seeds, and compare PSO with Random Search and Hill Climbing under an equal budget of 2,020 function evaluations. With $\omega = 0.7$, PSO reached a mean final fitness of 1.8×10^{-13} on Sphere and 2.1×10^{-7} on Rastrigin, and it was the only method to reach a fitness of 0.01 on Rastrigin. A low inertia weight gave the fastest convergence on Sphere but left one Rastrigin run in a local minimum, and a diagnostic run with a strong social pull showed swarm diversity collapsing around a suboptimal basin. The experiments are small, so these results illustrate rather than establish general behaviour.

1. Introduction

Many practical problems, from tuning a model's hyper-parameters to scheduling, can be written as finding the input that minimises a continuous objective function. When the function has no usable gradient or has many local minima, simple local search methods such as Hill Climbing can stop at the first basin they reach. Population-based methods try to avoid this by searching from many points at once.

PSO is one such method. Each particle remembers the best position it has visited, and the swarm shares the best position found by any particle; every move combines the particle's momentum with random pulls towards these two memories. The method is easy to implement and needs no gradient, which explains its wide use (Poli et al., 2007). Its behaviour, however, depends on how these pulls are weighted.

This report addresses the guiding question of the seminar: **how does PSO combine individual experience and social information to balance exploration and exploitation?** We focus on the role of the personal-best and global-best terms, the effect of the inertia weight, and the risk of premature convergence. The scope is deliberately small: two benchmark functions in two dimensions, a global-best PSO with velocity clamping, three inertia-weight settings with five random seeds each, and two baselines with the same evaluation budget. Section 2 reviews the two source papers, Section 3 defines the method and works through two numerical examples, Section 4 presents and discusses the results, and Section 5 concludes. Findings attributed to the papers are cited; all examples and experiments are our own.

2. Background and Paper Review

2.1 Origin of PSO and the original algorithm

PSO came out of simulations of social behaviour rather than from optimisation theory. Kennedy and Eberhart (1995) started from bird-flocking models and from E. O. Wilson's argument that members of a school of fish profit from the discoveries of the whole school when food is distributed unpredictably; they treated this sharing of information as the core hypothesis behind the method (pp. 1942–1943). Because they also wanted to model human social behaviour, which happens in an abstract space where two individuals can hold the same position, their agents were allowed to overlap and behave like points moving through a search space.

An early "cornfield" simulation gave each agent two memories: the best position it had found itself (pbest) and the best position found by any agent (gbest). When the authors removed parts that were not needed, random "craziness" and nearest-neighbour velocity matching could be dropped, but pbest and gbest were both necessary. A pull towards gbest much stronger than the pull towards pbest made the flock rush prematurely towards local minima, while the reverse made individuals wander; roughly equal pulls worked best (p. 1945). The final simplified version updates each velocity as

$$v \leftarrow v + 2 \cdot \text{rand}() \cdot (\text{pbest} - x) + 2 \cdot \text{rand}() \cdot (\text{gbest} - x),$$

where the factor 2 gives the random multiplier a mean of 1, so that agents overshoot their target about half the time (p. 1946). A version without the previous velocity was "quite ineffective at finding global optima". The reported evidence is small in scale: PSO trained a 13-weight XOR network to the error criterion in an average of 30.7 iterations with 20 agents, trained an Iris classifier as effectively as backpropagation (284 epochs on average), reached 92% test accuracy on an EEG dataset against 89% for backpropagation, and found the global optimum of the Schaffer f6 function in every run (pp. 1945, 1947). The authors called the generalisation result an "informal indication", and linked PSO's success to the particles' tendency to overshoot, which balances conservative testing of known regions against exploration of unknown ones (p. 1947).

2.2 Parameters and variants (Poli et al., 2007)

Poli, Kennedy and Blackwell (2007) restate the original algorithm with acceleration coefficients φ_1 and φ_2 and note that each velocity component was originally kept within $[-V_{max}, V_{max}]$ (Sect. 2.1). The value $\varphi_1 = \varphi_2 = 2$ used in early work made particle speeds grow without control, and a good V_{max} depended on the problem (Sect. 2.2). Two modifications took over the role of damping. The inertia weight ω of Shi and Eberhart multiplies the previous velocity; $1 - \omega$ acts like friction, so a high ω (around 0.9) favours exploration and a low ω (around 0.4) favours exploitation, and many studies decrease ω during a run (Sect. 2.3). Clerc's constriction coefficient scales the whole update and guarantees convergence without V_{max}; its common setting is algebraically equivalent to an inertia PSO with $\omega = 0.7298$ and $c_1 = c_2 \approx 1.49618$ (Sect. 2.4), close to our default $\omega = 0.7$ and $c_1 = c_2 = 1.5$.

The review also covers the communication structure of the swarm. In the gbest topology every particle is attracted by the best particle of the whole swarm; in the lbest ring each particle sees only its neighbours, which converges more slowly but is less vulnerable to local optima, and no topology was best on all functions (Sect. 3.1). Variants include the fully informed particle swarm, swarms with diversity control designed against the "tendency for the swarm to converge prematurely on local optima", and velocity-free bare-bones PSO (Sects. 2.5, 4.5, 4.6). Among about 700 application papers, large categories included image analysis, electricity networks and load dispatching, control, scheduling and neural networks (Sect. 6). Open questions include how to treat particles that leave the search space and a possible bias towards optima near the origin (Sect. 7).

2.3 Comparison of the two sources

The 1995 paper introduces a deliberately simple algorithm with fixed coefficients and a handful of demonstrations. The 2007 review shows that later work kept the core idea of pulls towards personal and swarm memories but added explicit control of the dynamics (ω or χ), studied topology, and collected evidence from a large body of applications. Both identify the same weakness: a swarm that follows its best member too strongly can converge before it finds the global optimum. Our experiments examine this trade-off.

3. Method and Worked Example

3.1 Problem and objective functions

We minimise a function f over the box $[-5.12, 5.12]^2$. The **fitness** of a candidate x is its objective value f(x); lower is better. Each call of f on one particle counts as one **function evaluation**, the cost used to compare algorithms. Two standard benchmarks with n = 2 are used:

- **Sphere:** $f(x) = \Sigma_i x_i^2$. One basin, minimum f = 0 at (0, 0); it tests convergence speed on a smooth landscape.
- **Rastrigin:** $f(x) = 10n + \Sigma_i (x_i^2 - 10 \cos 2\pi x_i)$. Same global minimum, but the cosine term creates a grid of local minima near integer coordinates; it tests whether the swarm gets trapped. Because it is computed as a difference of two numbers close to 20, values below about 10^{-13} are at floating-point resolution and are treated as the global minimum.

3.2 Algorithm

We use the global-best PSO with an inertia weight in the form of Poli et al. (2007, Eq. 2). For particle i at iteration t:

$$v_i(t+1) = \omega\, v_i(t) + c_1\, r_1 \otimes (pbest_i - x_i(t)) + c_2\, r_2 \otimes (gbest - x_i(t)), \qquad x_i(t+1) = x_i(t) + v_i(t+1)$$

The three terms are inertia (keeps part of the previous motion), the cognitive term (pull towards the particle's own best) and the social term (pull towards the swarm's best). The assignment sheet writes the position update with v(t); we use the newly computed velocity v(t+1), which is the form in Poli et al. (2007). Table 3.1 defines every symbol.

Table 3.1. Symbols and values used in this work.

Symbol	Meaning	Range or value used
$x_i(t)$	Position of particle i, a candidate solution	$[-5.12, 5.12]^2$
$v_i(t)$	Velocity of particle i (step taken in one iteration)	each component in [−2.0, 2.0]
$pbest_i$	Best position found so far by particle i	updated when f improves
gbest	Best position found so far by the whole swarm	updated every iteration
ω	Inertia weight: share of the previous velocity kept	0.4, 0.7 (default), 0.9
c_1, c_2	Cognitive and social acceleration coefficients	1.5 and 1.5
r_1, r_2	Uniform random numbers, new for every particle, dimension and iteration	U(0, 1)
⊗	Component-wise multiplication	
N, T	Number of particles, number of iterations	20 and 100 (5 and 5 in Sect. 3.4)
v_max	Velocity limit	2.0

The algorithm proceeds as follows:

1. Place N particles uniformly at random in the domain and draw velocities uniformly from [−v_max, v_max].
2. Evaluate f for every particle; set $pbest_i = x_i$ and gbest to the best pbest.
3. For t = 1, …, T: draw r_1 and r_2; update every velocity and clip it to [−v_max, v_max]; move every particle and clip it to the domain; evaluate f; replace $pbest_i$ when f improves; then update gbest.
4. Return gbest and f(gbest).

Velocity clamping follows the original PSO (Poli et al., 2007, Sect. 2.2); positions outside the domain are moved to the nearest boundary. All particles in one iteration use the gbest of the previous iteration. With N = 20 and T = 100, one run uses 20 + 20 × 100 = **2,020 function evaluations**.

3.3 Worked example 1: one update step in one dimension

We take $f(x) = x^2$, $\omega = 0.7$, $c_1 = c_2 = 1.5$, two particles and fixed random numbers, with no velocity limit. At iteration 0 each pbest equals the particle's own position, and gbest = −2 because $f(-2) = 4 < f(3) = 9$.

Table 3.2. One update step for two particles on $f(x) = x^2$.

Particle	x(0)	v(0)	r_1	r_2	Inertia ωv	Cognitive	Social c_2r_2(gbest − x)	v(1)	x(1)	f(x(1))
1	3	0.5	0.4	0.6	0.35	0	1.5 × 0.6 × (−5) = −4.5	−4.15	−1.15	1.3225
2	−2	−0.5	0.8	0.3	−0.35	0	0	−0.35	−2.35	5.5225

The cognitive term is zero for both particles because pbest = x at iteration 0. Particle 2 is the current gbest, so its social term is also zero and inertia alone moves it further from the optimum (f rises from 4 to 5.5225). Particle 1 is pulled strongly towards particle 2, overshoots the optimum and lands at −1.15; its fitness falls from 9 to 1.3225, so $pbest_1 = -1.15$ and the new gbest = −1.15. One particle can get worse while the swarm's best improves.

3.4 Worked example 2: five particles over five iterations

We ran the full algorithm on the 2-D Sphere function with N = 5, T = 5, seed 42 and the default parameters. Table 3.3 gives every particle's state at iterations 0 and 1.

Table 3.3. Particle states at iterations 0 and 1 (Sphere, seed 42). At iteration 0, $pbest_i = x_i$; at iteration 1, every particle improved, so $pbest_i = x_i$ again.

Particle	$x_1(0)$	$x_2(0)$	$v_1(0)$	$v_2(0)$	f(0)	$x_1(1)$	$x_2(1)$	$v_1(1)$	$v_2(1)$	f(1)
1	2.8053	−0.6259	−0.5168	1.7071	8.2615	2.4435	0.5691	−0.3618	1.1949	6.2947
2	3.6720	2.0210	0.5755	1.2910	17.5685	3.6513	1.4539	−0.0208	−0.5671	15.4455
3	−4.1556	4.8704	−0.2263	−1.0910	40.9897	−2.1556	2.8704	2.0000	−2.0000	12.8858
4	2.6741	2.9293	0.2183	−1.7447	15.7314	2.8525	0.9293	0.1784	−2.0000	9.0003
5	−3.8081	−0.5080	1.3105	0.5267	14.7599	−1.8081	−0.2578	2.0000	0.2503	3.3357
gbest	2.8053	−0.6259			8.2615	−1.8081	−0.2578			3.3357

Particle 2, step by step. The random numbers used were r_1 = (0.9707, 0.8931) and r_2 = (0.3258, 0.3705). In dimension 1, the inertia term is 0.7 × 0.5755 = 0.4028, the cognitive term is 0 and the social term is 1.5 × 0.3258 × (2.8053 − 3.6720) = −0.4236, so $v_1(1) = -0.0208$ and $x_1(1) = 3.6513$. In dimension 2, inertia gives 0.9037 and the social term 1.5 × 0.3705 × (−0.6259 − 2.0210) = −1.4709, so $v_2(1) = -0.5671$ and $x_2(1) = 1.4539$, with f = 15.4455. Particles 3, 4 and 5 hit the velocity limit: particle 3 had a raw velocity of (4.7444, −2.3258), clipped to (2.0, −2.0). Over five iterations f(gbest) fell from 8.2615 to 3.3357, 0.1734, 0.1734, 0.1734 and 0.0195 (Figure 3.1).

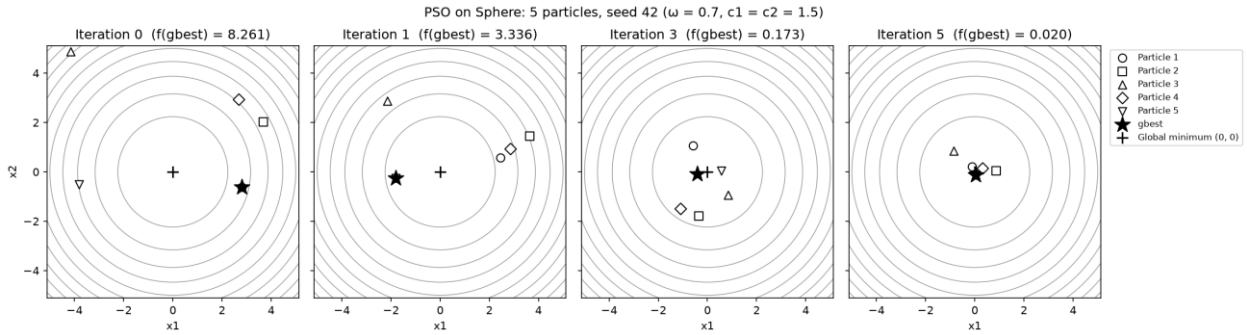


Figure 3.1. Five particles on the Sphere contours at iterations 0, 1, 3 and 5 (seed 42). The star marks gbest and the cross the global minimum. Source: team experiment.

3.5 Experimental setup

To study the inertia weight we compare **A** ($\omega = 0.4$), **B** ($\omega = 0.7$) and **C** ($\omega = 0.9$), all with $c_1 = c_2 = 1.5$, $N = 20$ and $T = 100$, on both functions with seeds 0 to 4. For the comparison with local search, Random Search samples uniformly from the domain, and Hill Climbing starts from one uniform point, adds Gaussian noise with $\sigma = 0.3$ per coordinate, clips to the domain and accepts only strict improvements, without restarts. Both baselines receive exactly 2,020 evaluations per run, counting initialisation and rejected proposals, and use the same seeds. We report the final best fitness (mean, sample standard deviation, best and worst over five seeds) and the number of evaluations until the best fitness first falls below 0.01. Each run is fully determined by its seed; the code and raw results are in the team's notebooks.

4. Results and Discussion

4.1 Equal-budget comparison

Table 4.1. Final best fitness after 2,020 evaluations (five seeds; lower is better).

Function	Method	Mean	SD	Best	Worst
Sphere	PSO ($\omega = 0.7$)	1.795e−13	1.843e−13	4.595e−14	4.616e−13
Sphere	Random Search	1.397e−02	1.605e−02	1.216e−03	4.154e−02
Sphere	Hill Climbing	2.602e−04	2.883e−04	4.341e−05	6.466e−04
Rastrigin	PSO ($\omega = 0.7$)	2.077e−07	4.565e−07	5.684e−14	1.024e−06
Rastrigin	Random Search	1.232e+00	8.268e−01	2.408e−01	2.484e+00
Rastrigin	Hill Climbing	5.205e+00	4.557e+00	1.991e+00	1.304e+01

Table 4.2. Evaluations until the best fitness first falls below 0.01 (mean over the seeds that reached it). PSO is recorded at the end of each 20-evaluation batch.

Function	Method	Mean evaluations	Seeds reaching 0.01
Sphere	PSO ($\omega = 0.7$)	132.0	5/5
Sphere	Random Search	659.0	3/5
Sphere	Hill Climbing	42.6	5/5
Rastrigin	PSO ($\omega = 0.7$)	784.0	5/5
Rastrigin	Random Search	not reached	0/5
Rastrigin	Hill Climbing	not reached	0/5

On Sphere, Hill Climbing reached 0.01 in all five runs after a mean of 42.6 evaluations, compared with 132.0 for PSO (Table 4.2). Its progress then slowed, while PSO kept improving (Figure 4.1) and finished with a mean of 1.795×10^{-13}, against 2.602×10^{-4} for Hill Climbing and 1.397×10^{-2} for Random Search (Table 4.1). A single local search gave rapid initial progress on the smooth objective; PSO was stronger when the goal was final accuracy under this budget.

On Rastrigin, PSO reached 0.01 in all five runs after 784 evaluations on average, and neither baseline reached it. Random Search finished ahead of Hill Climbing (means 1.232 and 5.205), while PSO averaged 2.077×10^{-7}. Uniform sampling kept trying points in different basins, whereas the single-start Hill Climbing, which accepts only improvements with a fixed step scale, rarely left the basin it reached first. The comparison measures fitness against evaluations; runtime was outside the scope of the experiment.

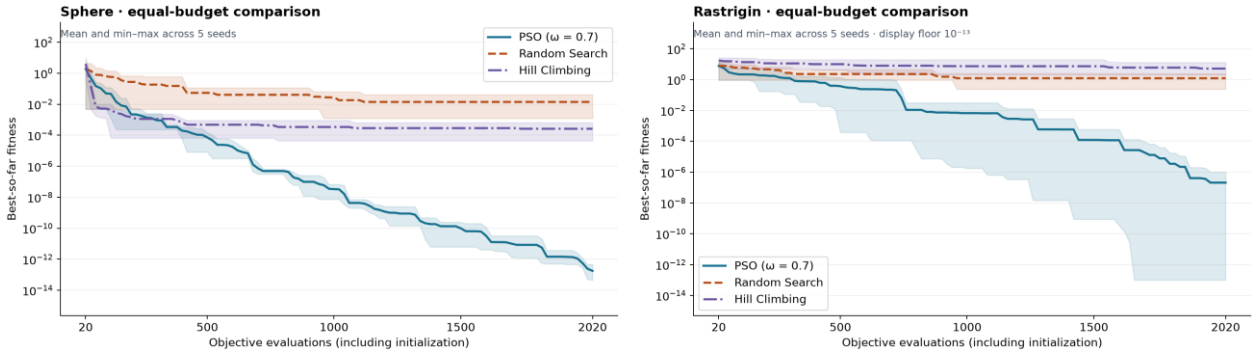


Figure 4.1. Best-so-far fitness against function evaluations on Sphere (left) and Rastrigin (right): mean and min–max range over five seeds. The Rastrigin plot uses a display floor of 10^{-13}. Source: team experiments.

4.2 Sensitivity to the inertia weight

Table 4.3. Mean final PSO fitness for three inertia weights (five seeds, 2,020 evaluations, $c_1 = c_2 = 1.5$).

Function	A (ω = 0.4)	B (ω = 0.7)	C (ω = 0.9)
Sphere	5.455e−31	1.795e−13	4.158e−05
Rastrigin	1.990e−01	2.077e−07	8.452e−03

The lowest inertia gave the best final mean on Sphere, while the middle setting gave the best mean on Rastrigin (Table 4.3). On Sphere there is only one basin, so strong damping lets the swarm settle and refine quickly; with ω = 0.9 the swarm was still converging when the budget ran out (mean 4.2×10^{-5}). On Rastrigin, four runs with ω = 0.4 reached values below 10^{-6}, but the fifth stopped near 0.995, a local minimum next to the origin, and this single run dominates the mean. This matches the interpretation of $1 - \omega$ as friction in Poli et al. (2007): low ω favours exploitation, which pays off on a unimodal function and becomes risky on a multimodal one. Only ω was varied, so the sweep shows sensitivity to ω under these settings; ranking the effects of c_1 and c_2 would need a separate controlled sweep.

4.3 Premature convergence

In a separate Rastrigin diagnostic we ran five particles with ω = 0.4, $c_1 = 0.5$ and $c_2 = 2.5$ for 100 iterations over seeds 0 to 19. Seed 0 was the first to finish above 0.9. It ended at a verified local minimum near (−0.994959, 0.994959) with fitness 1.989918, while the global value is 0. The mean distance of the particles to the swarm centroid fell from 4.184 to 7.85×10^{-10}, so the swarm had collapsed onto one point. The default run with the same seed kept a final mean distance of 0.0918 and found a point with fitness 1.408×10^{-8} (Figure 4.2).

A strong pull towards gbest, weak momentum and a small population are plausible contributors to this rapid loss of diversity. This mirrors the 1995 observation that a relatively high g_increment makes the flock rush prematurely towards local minima. The diagnostic changes several settings at once and uses 505 evaluations against 2,020 for the default run, so it shows how a swarm can converge tightly around a suboptimal basin without measuring how much each parameter contributed.

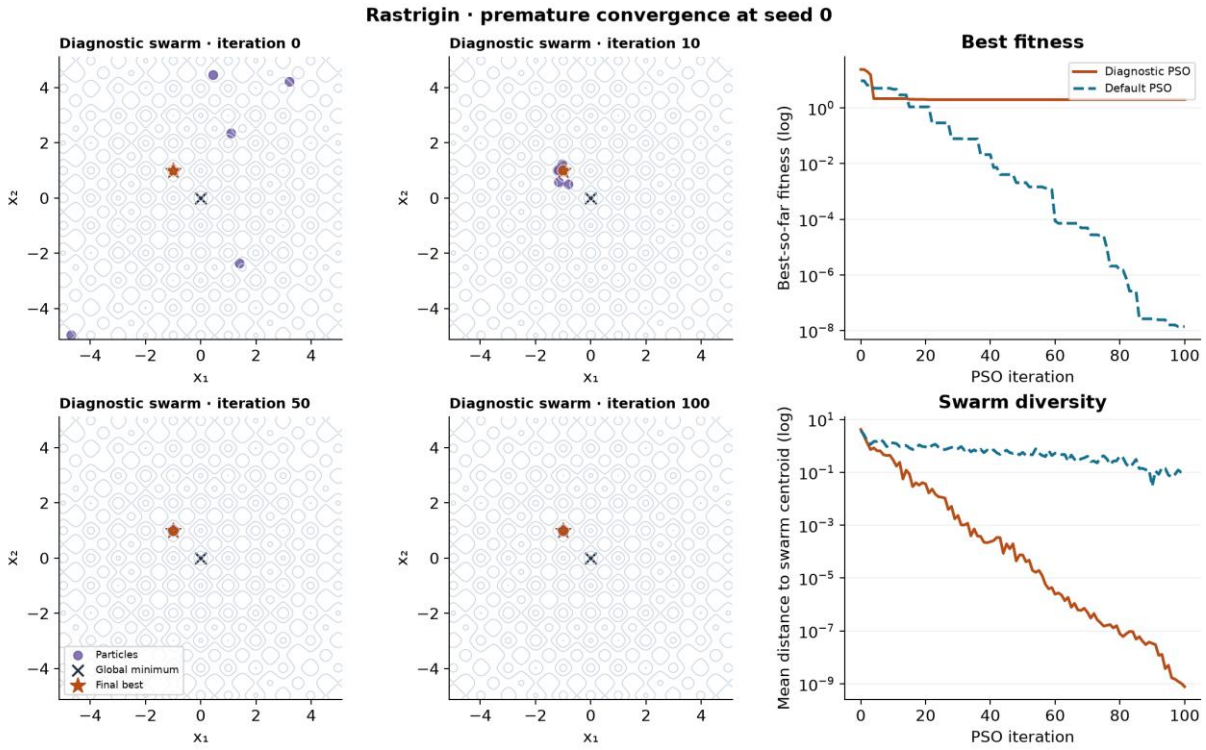


Figure 4.2. Premature convergence on Rastrigin (seed 0): diagnostic swarm at iterations 0, 10, 50 and 100, with best fitness and swarm diversity for the diagnostic and default runs. Source: team experiment.

4.4 Applications, strengths and limitations

Poli et al. (2007, Sect. 6) list scheduling, neural networks and electricity load dispatching among the main PSO application areas. Examples are the tuning of convolutional-network hyper-parameters, where each evaluation requires training and validating a network (Wang et al., 2019), and economic dispatch of generators under operating constraints (Gaing, 2003). Our results suggest PSO is worth testing when final solution quality matters and many candidates can be evaluated within the budget; Figure 4.2 shows why diversity matters on multimodal objectives.

The evidence has clear limits. We used two functions in two dimensions, five seeds per setting, one Hill Climbing step scale and a single topology. Both functions have their global minimum at the centre of the domain, and Poli et al. (2007, Sect. 7.1) mention a possible bias of PSO towards optima near the origin, so shifted versions of the functions would be a fairer test. The results therefore illustrate PSO's behaviour; they do not establish the same ranking for other problems or for the applications above.

5. Conclusion

PSO balances exploration and exploitation through what each particle remembers and what the swarm shares. Personal bests keep information from each particle's own search and maintain diversity; the global best lets the swarm refine a promising region together; and the inertia weight decides how much motion carries the particles past the memories they are attracted to. In our benchmarks the default setting ($\omega = 0.7$, $c_1 = c_2 = 1.5$) used this balance to finish with a much better mean fitness than Random Search and Hill Climbing under the same budget. When the balance tilts towards exploitation, through low inertia, a strong social pull or few particles, the swarm can collapse onto a suboptimal basin, as our Rastrigin diagnostic showed. A controlled follow-up could vary the population size and the acceleration coefficients one at a time under a fixed budget, use shifted and higher-dimensional functions, and compare the gbest topology with an lbest ring or diversity-triggered restarts.

References

Gaing, Z.-L. (2003). Particle swarm optimization to solving the economic dispatch considering the generator constraints. *IEEE Transactions on Power Systems, 18*(3), 1187–1195. https://doi.org/10.1109/TPWRS.2003.814889

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In *Proceedings of ICNN'95, International Conference on Neural Networks* (Vol. 4, pp. 1942–1948). IEEE. https://doi.org/10.1109/ICNN.1995.488968

Poli, R., Kennedy, J., & Blackwell, T. (2007). Particle swarm optimization: An overview. *Swarm Intelligence, 1*(1), 33–57. https://doi.org/10.1007/s11721-007-0002-0

Wang, Y., Zhang, H., & Zhang, G. (2019). cPSO-CNN: An efficient PSO-based algorithm for fine-tuning hyper-parameters of convolutional neural networks. *Swarm and Evolutionary Computation*, *49*, 114–123. https://doi.org/10.1016/j.swevo.2019.06.002

Use of AI tools

Claude (Anthropic) and OpenAI Codex were used to draft reading notes, implement and run the experiment notebooks, and draft sections of this report and the slides. Every number in this report was reproduced from the team's code and data, quotations and page references were checked against the source papers, and the text was reviewed by the team. Prompts and checks are recorded in the team's AI-use log.